Apuntes de Compiladores: Árboles Sintácticos y el Conjunto Follow

Material de Estudio

1 El Árbol de Sintaxis Abstracta (AST)

Para entender las reglas gramaticales, primero debemos entender qué busca el compilador. El código fuente es, para la computadora, una simple cadena plana de caracteres (por ejemplo, `a + b`). No tiene jerarquía ni significado por sí sola.

El trabajo del analizador sintáctico (Parser) es validar si esta cadena sigue las reglas de la gramática. Sin embargo, no basta con decir "es válido". Durante este proceso, se ejecutan **acciones semánticas** que se encargan de construir el **Árbol de Sintaxis Abstracta (AST)**.

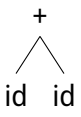
El AST elimina el "ruido" (como los paréntesis, puntos y comas, o las reglas intermedias de la gramática) y deja solo la esencia de la operación, preparada para que el analizador semántico revise tipos y el generador de código cree las instrucciones.

1.1 Ejemplo Práctico

Imaginemos la siguiente gramática básica para sumas:

$$E \to E + T \mid T$$
$$T \to \text{id}$$

Si tenemos la cadena `id + id`, el árbol de derivación (Parse Tree) incluiría nodos para E y T. Pero el **AST** generado por las rutinas semánticas es más concreto:



Este árbol nos dice: "Hay una operación de suma, cuyo hijo izquierdo es un identificador y su hijo derecho es otro identificador". Con este árbol en mente, es más fácil entender las fronteras de cada nodo y, por ende, el conjunto Follow.

2 ¿Qué es Follow y de dónde derivan sus 3 reglas?

El conjunto Follow(A) responde a una pregunta crítica durante el parsing: *"¿Qué tokens (terminales) tienen el derecho legal de aparecer inmediatamente a la derecha del no terminal A durante una derivación válida?"*.

Veamos el origen lógico de las tres reglas fundamentales.

2.1 Regla 1: La raíz y el fin de cadena

La Regla: Si S es el símbolo inicial de la gramática, entonces el símbolo de fin de cadena $\$$ se añade a Follow(S).

El Origen: Imagina el AST completo de un programa. El símbolo S es el nodo raíz absoluto que engloba todo el código. Cuando el analizador termina de construir este nodo raíz, ha procesado el 100% del programa. Si hubiera algo más escrito a la derecha, sería código sobrante o un error sintáctico. La única señal legal que indica que la raíz ha terminado exitosamente es el marcador de fin de archivo ($). Por decreto, el fin de cadena es el único "vecino" válido de la raíz.

2.2 Regla 2: Los hermanos derechos

La Regla: Si existe una producción $A \to \alpha B \beta$, entonces todo lo que esté en First(β) (excepto ϵ) se añade a Follow(B).

El Origen: Observa la regla de producción como ramas de un árbol. Al expandir A, los nodos α, B y β se convierten en hermanos de izquierda a derecha. Físicamente, el subárbol generado por β está plantado inmediatamente a la derecha de B. Por lo tanto, la primera hoja (el primer terminal) que produzca β, será exactamente el símbolo que le esté "tocando la espalda" a B. Excluimos ϵ (la cadena vacía) porque la "nada" no es un token que el escáner pueda leer.

2.3 Regla 3: La herencia del padre (el último hijo)

La Regla: Si existe una producción $A \to \alpha B$, o una producción $A \to \alpha B \beta$ donde First(β) contiene a ϵ, entonces todo Follow(A) se añade a Follow(B).

El Origen: ¿Qué pasa si B no tiene ningún hermano a su derecha? En $A \to \alpha B$, el nodo B es el último hijo del nodo A. Al terminar de procesar B, terminamos de procesar A. Como B está en el borde derecho absoluto de su padre, su frontera exterior es la misma frontera de A. Lo que sea que estuviera esperando pacientemente a la derecha de A en el nivel superior del árbol, quedará inevitablemente pegado a B. El caso $A \to \alpha B \beta$ aplica la misma lógica si β desaparece (se vuelve ϵ). Al esfumarse el hermano derecho, B queda nuevamente expuesto al borde de su padre A y hereda sus seguidores.

3 Pseudocódigo para calcular Follow

Para implementar esto en un compilador, el algoritmo iterativo clásico es el siguiente:

```
Para cada no terminal NT de la gramática:
    Inicializar Follow(NT) como un conjunto vacío

Agregar "$" a Follow(Símbolo_Inicial)

Hacer:
    Para cada producción A -> X1 X2 ... Xn:
        Para cada símbolo Xi de la producción:
            Si Xi es un No Terminal:
                // Regla 2: Mirar al hermano derecho
                Sea Beta = X_{i+1} ... Xn
                Agregar First(Beta) excluyendo epsilon a Follow(Xi)

                // Regla 3: Herencia si está al final o Beta es anulable
                Si Beta no existe (Xi es el último símbolo)
                   O si epsilon está en First(Beta):
```

```
                    Agregar todo Follow(A) a Follow(Xi)
Mientras se hayan añadido nuevos elementos a algún conjunto Follow
```

4 Ejemplos Resueltos

4.1 Ejercicio básico

Gramática:

$$
\begin{aligned}
&1.\ S \to AB \\
&2.\ A \to a \\
&3.\ B \to b
\end{aligned}
$$

Cálculo:

- **Follow(S):** Por la Regla 1, agregamos $\{\$\}$.
- **Follow(A):** De la regla 1 ($S \to AB$), el hermano derecho de A es B. Agregamos First(B). Como First$(B) = \{b\}$, entonces Follow$(A) = \{b\}$. (Regla 2).
- **Follow(B):** De la regla 1 ($S \to AB$), B está al final de la producción. Hereda el Follow de su padre S. Follow$(B) = \{\$\}$. (Regla 3).

4.2 Con ϵ

Gramática:

$$
\begin{aligned}
&1.\ S \to ABA \\
&2.\ A \to aA \mid \epsilon \\
&3.\ B \to bB \mid c
\end{aligned}
$$

Firsts previos necesarios: First$(A) = \{a, \epsilon\}$, First$(B) = \{b, c\}$.
Cálculo:

- **Follow(S):** Solo $\{\$\}$ por ser el inicio.
- **Follow(A):**
  - De $S \to ABA$, el *primer* A es seguido por B. Agregamos First$(B) = \{b, c\}$.
  - El *segundo* A está al final de S. Hereda Follow$(S) = \{\$\}$.
  - De $A \to aA$, el A final hereda de sí mismo (no aporta nada nuevo).
  - **Resultado:** Follow$(A) = \{b, c, \$\}$.
- **Follow(B):**
  - De $S \to ABA$, B es seguido por A. Agregamos First(A) excepto $\epsilon \to \{a\}$.
  - Como A es anulable ($\epsilon \in$ First(A)), B puede quedar expuesto al final. Hereda Follow$(S) = \{\$\}$.
  - **Resultado:** Follow$(B) = \{a, \$\}$.

4.3 Alta recursión y ciclos múltiples

Gramática:

$$
\begin{aligned}
&1.\ S \to XYZ \\
&2.\ X \to x \mid \epsilon \\
&3.\ Y \to y \mid \epsilon \\
&4.\ Z \to Sa \mid z
\end{aligned}
$$

Análisis de Firsts:

- First$(X) = \{x, \epsilon\}$
- First$(Y) = \{y, \epsilon\}$
- Para First(Z) y First(S) hay una recursión:
  - First(S) = First(XYZ). Como X e Y son anulables, el First "atraviesa" la cadena hasta Z. Así que First$(S) = \{x, y\} \cup$ First(Z).
  - First$(Z) = \{z\} \cup$ First$(Sa) = \{z\} \cup$ First(S).
  - Uniendo ambos hechos: Ambos comparten los mismos tokens. First(S) = First$(Z) = \{x, y, z\}$.

Cálculo Iterativo de Follow:

- **Iteración 1 (Regla 1 y Reglas Directas):**
  - Follow(S) recibe $\{\$\}$ (Regla 1). De $Z \to Sa$, S es seguido por a, así que Follow$(S) = \{\$, a\}$.
  - En $S \to XYZ$:
    - El seguidor de X es First(YZ). Como Y puede ser ϵ, es First$(Y) \setminus \{\epsilon\} \cup$ First$(Z) = \{y\} \cup \{x, y, z\} = \{x, y, z\}$.
    - El seguidor de Y es First$(Z) = \{x, y, z\}$.
- **Iteración 2 (Reglas de Herencia):**
  - ¿X hereda de S? No, porque aunque Y es anulable, Z **no** lo es. Hay un muro de tokens (x, y o z) garantizado después de X.
  - ¿Y hereda de S? No, por la misma razón: Z no puede desaparecer.
  - ¿Z hereda de S? Sí, Z está en el extremo derecho de $S \to XYZ$. Por tanto, Z recibe todo Follow(S).
  - Así que Follow$(Z) = \{\$, a\}$.

Resultado Final del Nivel Difícil:

- Follow$(S) = \{\$, a\}$
- Follow$(X) = \{x, y, z\}$
- Follow$(Y) = \{x, y, z\}$
- Follow$(Z) = \{\$, a\}$

Este último ejemplo demuestra cómo la propiedad anulable (ϵ) propaga los conjuntos no solo en *First*, sino que dictamina si se aplica la Regla 3 de *Follow*, evaluando la secuencia entera de los hermanos.

4.4 Dependencias cíclicas y Punto Fijo

Una situación clásica en gramáticas recursivas es que la Regla 3 genere un ciclo de herencia. Por ejemplo, calcular Follow(S) requiere Follow(A), que requiere Follow(B), que a su vez requiere Follow(S). En diseño de compiladores esto se resuelve mediante el concepto de **iteración de punto fijo**.

Como las reglas dictan que los conjuntos sólo pueden crecer, el algoritmo inicializa todo vacío e itera aplicando las tres reglas. Los conjuntos actúan como vasos comunicantes. El proceso no colapsa en un bucle infinito; simplemente se repiten las pasadas hasta alcanzar el "punto fijo": una iteración completa donde ningún conjunto Follow aumenta su tamaño. En ese momento, la evaluación termina.

Gramática:

$$1.\ S \to B$$
$$2.\ B \to A$$
$$3.\ A \to S \mid c$$

Resolución paso a paso:

Pasada 1: Iniciamos la raíz. Por la Regla 1, el fin de cadena entra al símbolo inicial: Follow$(S) = \{\$\}$

Revisamos la producción 1 ($S \to B$). Por la Regla 3, B es el último símbolo, así que hereda del padre S: Follow$(B) =$ Follow$(B) \cup$ Follow$(S) \implies \{\$\}$

Revisamos la producción 2 ($B \to A$). Por la Regla 3, A es el último símbolo, hereda del padre B: Follow$(A) =$ Follow$(A) \cup$ Follow$(B) \implies \{\$\}$

Revisamos la producción 3 ($A \to S$). Por la Regla 3, S hereda del padre A: Follow$(S) =$ Follow$(S) \cup$ Follow$(A) \implies \{\$\}$

Pasada 2: Se vuelve a evaluar todo de arriba a abajo. Al cruzar los datos de S, B y A, observamos que todos ya poseen el símbolo $\$$. Ninguna regla aporta un terminal nuevo. Al no haber modificaciones, se declara el punto fijo y el algoritmo termina.

Resultado: Follow$(S) = \{\$\}$
Follow$(B) = \{\$\}$
Follow$(A) = \{\$\}$